individual. We also define a function that will create the subsequent generation.

Listing 6.3. Evolving strings: evaluate individuals and create new generation

```
def evaluate_population(pop, target):                          1
    avg_fit = 0
    for i in range(len(pop)):
        fit = similar(pop[i].string, target)
        pop[i].fitness = fit
        avg_fit += fit
    avg_fit /= len(pop)
    return pop, avg_fit

def next_generation(pop, size=100, length=26, mut_rate=0.01):   2
    new_pop = []
    while len(new_pop) < size:
        parents = random.choices(pop,k=2, weights=[x.fitness for x
in pop])
        offspring_ = recombine(parents[0],parents[1])
        child1 = mutate(offspring_[0], mut_rate=mut_rate)
        child2 = mutate(offspring_[1], mut_rate=mut_rate)
        offspring = [child1, child2]
        new_pop.extend(offspring)
    return new_pop
```

- ***1*** Assigns a fitness score to each individual in the population
- ***2*** Generates a new generation by recombination and mutation

These are the last two functions we need to complete the evolutionary process. We have a function that evaluates each individual in the population and assigns a fitness score, which just indicates how similar the individual's string is to the target string. The fitness score will vary depending on what the objective is for a given problem. Lastly, we have a function that generates a new population by sampling the most fit individuals in the current population, recombining them to produce offspring, and mutating them.

In listing 6.4 we put everything together and iterate the previous steps to some maximum number of generations. That is, we start with an initial population, go through the process of fitness-scoring individuals and creating a new offspring population, and then repeat this sequence a number of times. After a sufficient number of generations, we expect the final population to be enriched with strings very close to our target string.